DİZİLER / LİSTELER — DERS NOTU

1. GİRİŞ

Programlarda çoğu zaman tek bir değer saklamak yeterli olmaz. Örneğin bir sınıftaki 30 öğrencinin notunu, bir listedeki 100 ürünün fiyatını veya bir metindeki tüm kelimeleri saklamak gerekebilir. Bu tür durumlarda her değer için ayrı bir değişken tanımlamak hem pratik değildir hem de sürdürülemez. İşte bu noktada **diziler (arrays)** ve **listeler (lists)** devreye girer. Bu bölümde veri tutma mantığı, dizi ve liste kavramları, temel liste işlemleri ve C++ dili üzerinden uygulamalar ele alınacaktır.

2. VERİ TUTMA MANTIĞI

2.1 Neden Diziye İhtiyaç Duyulur?

Tek bir değişken yalnızca bir değer saklayabilir. Birden fazla değeri saklamak için iki yol vardır:

Yöntem 1: Ayrı Değişkenler (Kötü Yaklaşım)

```
int not1 = 85;
int not2 = 92;
int not3 = 78;
int not4 = 65;
int not5 = 90;
// 30 öğrenci için 30 değişken gerekir
```

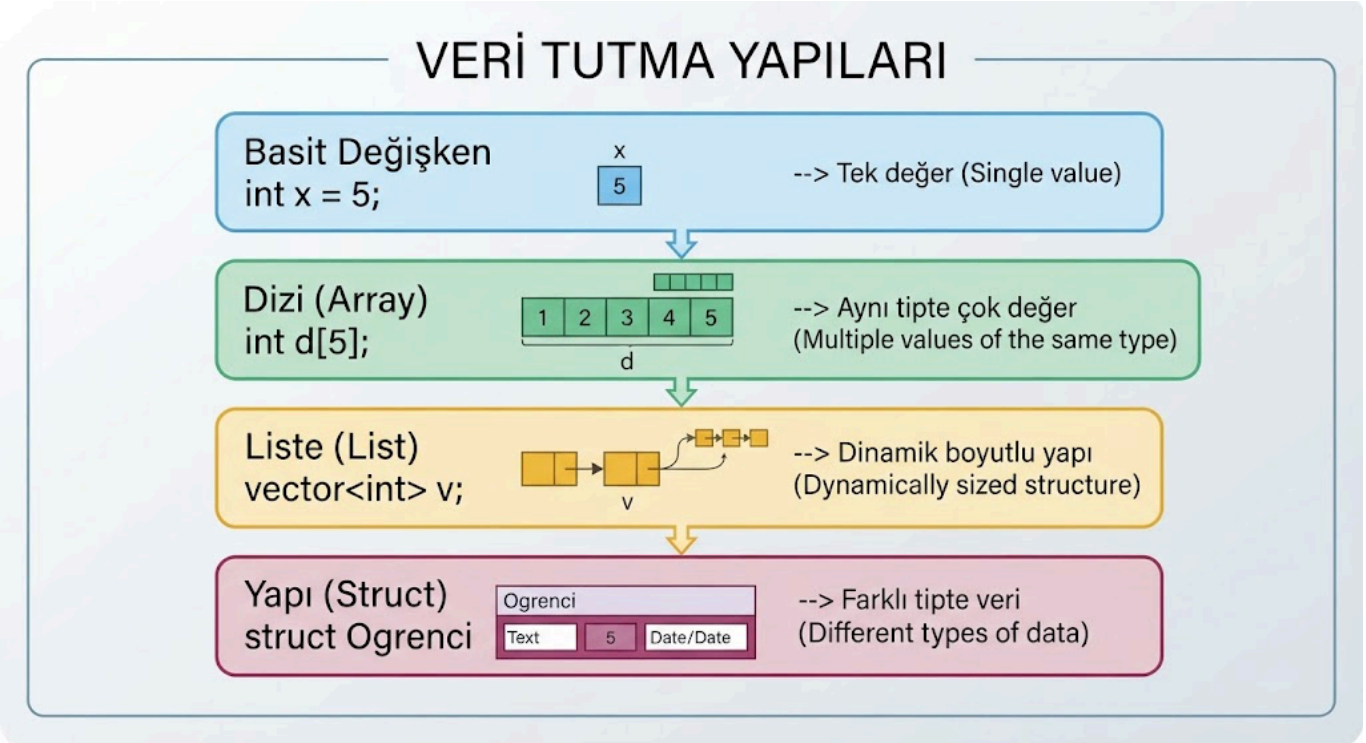
Yöntem 2: Dizi Kullanımı (İyi Yaklaşım)

```
int notlar[30];  // 30 öğrencinin notu tek bir yapıda
```

Karşılaştırma:



2.2 Veri Tutma Yapıları



2.3 Dizi ve Liste Kavramları

Dizi (Array): Aynı tipte verilerin bellekte **ardışık** olarak saklandığı, **sabit boyutlu** yapıdır.

Liste (List / Vector): Aynı tipte verilerin saklandığı, **dinamik boyutlu** (büyüyebilen/küçülebilen) yapıdır.

```
+----------------------+----------------------+----------------------+
| Özellik              | Dizi (Array)         | Liste (Vector)       |
+----------------------+----------------------+----------------------+
```

```
| Boyut                | Sabit                | Dinamik              |
| Bellek               | Ardışık              | Ardışık (genelde)    |
| Boyut değiştirme     | Mümkün değil         | Mümkün               |
| Ekleme/Silme         | Zor                  | Kolay                |
| Erişim hızı          | Çok hızlı            | Hızlı                |
| Bellek verimliliği   | Yüksek               | Biraz düşük          |
| Kullanım             | Sabit boyutlu veri   | Değişken boyutlu veri|
+----------------------+----------------------+----------------------+
```

---

3. DİZİLER (ARRAYS)

3.1 Dizi Tanımlama

Genel Yapı:

```
veri_tipi dizi_adi[boyut];
```

Örnekler:

```
int sayilar[5];           // 5 tam sayı
double notlar[30];        // 30 ondalıklı sayı
char harfler[10];         // 10 karakter
string isimler[20];       // 20 string
```

3.2 Dizi Başlatma (Initialization)

Yöntem 1: Tek Tek Atama

```
int sayilar[5];
sayilar[0] = 10;
sayilar[1] = 20;
sayilar[2] = 30;
sayilar[3] = 40;
sayilar[4] = 50;
```

Yöntem 2: Tanımlama Sırasında Başlatma

```
int sayilar[5] = {10, 20, 30, 40, 50};
```

Yöntem 3: Boyut Belirtmeden Başlatma

```
int sayilar[] = {10, 20, 30, 40, 50};  // Boyut otomatik 5 olur
```

Yöntem 4: Kısmi Başlatma

```
int sayilar[5] = {10, 20};  // Kalan elemanlar 0 olur
// sayilar = {10, 20, 0, 0, 0}
```

Yöntem 5: Tümünü Sıfırlama

```
int sayilar[5] = {0};  // Tüm elemanlar 0
```

3.3 Dizi Elemanlarına Erişim

Dizi elemanlarına **indeks** ile erişilir. İndeksler **0'dan** başlar.



Erişim Örnekleri:

```
int sayilar[5] = {10, 20, 30, 40, 50};

cout << sayilar[0];   // 10
cout << sayilar[2];   // 30
cout << sayilar[4];   // 50
```

```
sayilar[1] = 25;      // 2. elemanı 25 yap
cout << sayilar[1];   // 25
```

Dikkat: İndeks 0'dan başlar, bu nedenle 5 elemanlı bir dizinin geçerli indeksleri **0, 1, 2, 3, 4**'tür. `sayilar[5]`'e erişmek **hata**dır (dizi sınırları dışına çıkma).

3.4 Dizi Boyutunu Bulma

```
int sayilar[] = {10, 20, 30, 40, 50};

int boyut = sizeof(sayilar) / sizeof(sayilar[0]);
// boyut = 20 / 4 = 5
```

3.5 Dizilerde Döngü Kullanımı

For Döngüsü ile Dizi Elemanlarını Yazdırma:

```
#include <iostream>
using namespace std;

int main() {
    int sayilar[5] = {10, 20, 30, 40, 50};

    for (int i = 0; i < 5; i++) {
        cout << "sayilar[" << i << "] = " << sayilar[i] << endl;
    }

    return 0;
}
```

Çıktı:

```
sayilar[0] = 10
sayilar[1] = 20
sayilar[2] = 30
sayilar[3] = 40
sayilar[4] = 50
```

While Döngüsü ile:

```
int i = 0;
while (i < 5) {
    cout << sayilar[i] << " ";
```

```
    i++;
}
```

Range-based For Döngüsü (C++11):

```
for (int eleman : sayilar) {
    cout << eleman << " ";
}
```

3.6 Dizi İşlemleri

Toplam Hesaplama:

```
int toplam = 0;
for (int i = 0; i < 5; i++) {
    toplam += sayilar[i];
}
cout << "Toplam: " << toplam << endl;
```

Ortalama Hesaplama:

```
double ortalama = (double)toplam / 5;
cout << "Ortalama: " << ortalama << endl;
```

En Büyük Elemanı Bulma:

```
int en_buyuk = sayilar[0];
for (int i = 1; i < 5; i++) {
    if (sayilar[i] > en_buyuk) {
        en_buyuk = sayilar[i];
    }
}
cout << "En büyük: " << en_buyuk << endl;
```

En Küçük Elemanı Bulma:

```
int en_kucuk = sayilar[0];
for (int i = 1; i < 5; i++) {
    if (sayilar[i] < en_kucuk) {
        en_kucuk = sayilar[i];
    }
```

```
}
cout << "En küçük: " << en_kucuk << endl;
```

Arama:

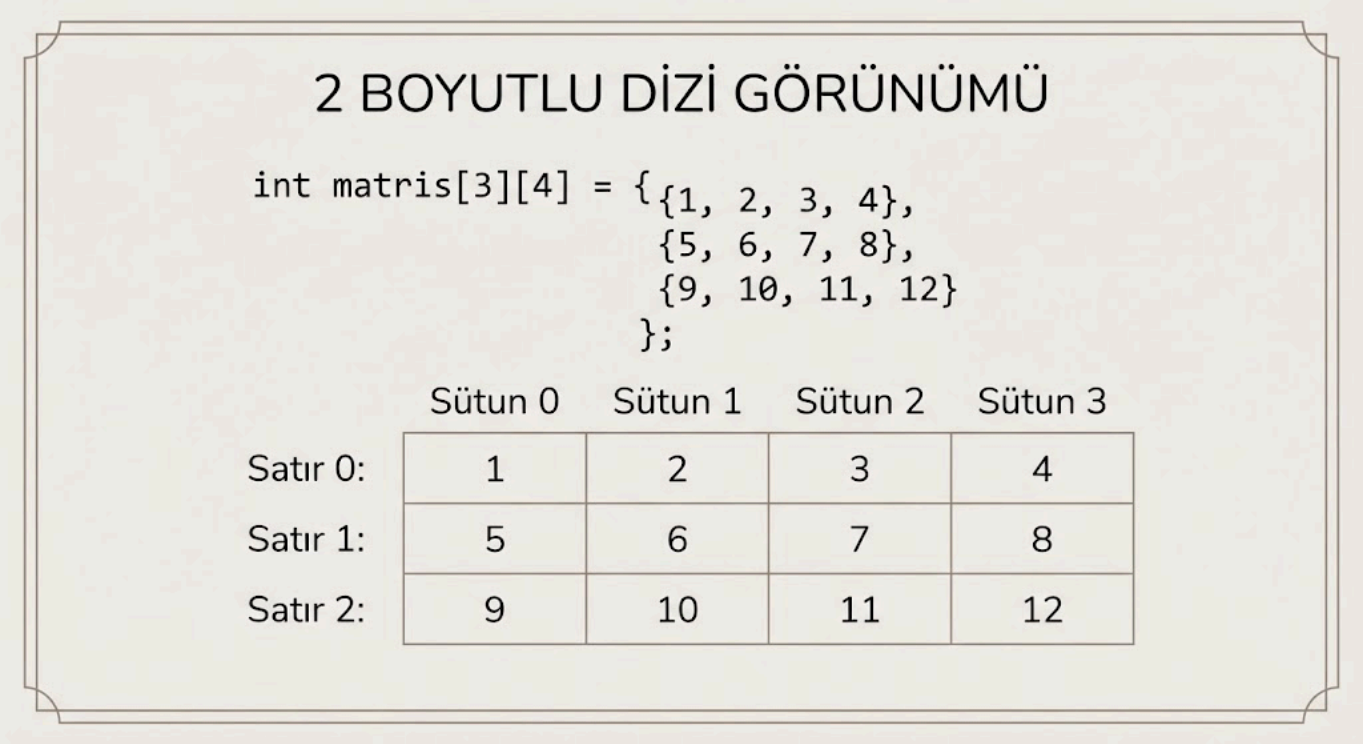
```
int aranan = 30;
bool bulundu = false;
for (int i = 0; i < 5; i++) {
    if (sayilar[i] == aranan) {
        cout << "Eleman " << i << ". indekste bulundu." << endl;
        bulundu = true;
        break;
    }
}
if (!bulundu) {
    cout << "Eleman bulunamadı." << endl;
}
```

3.7 Çok Boyutlu Diziler

İki Boyutlu Dizi (Matris):

```
int matris[3][4] = {
    {1, 2, 3, 4},
    {5, 6, 7, 8},
    {9, 10, 11, 12}
};
```

Bellek Görünümü:



Erişim:

```
cout << matris[0][0];  // 1
cout << matris[1][2];  // 7
cout << matris[2][3];  // 12
```

İç İçe Döngü ile Yazdırma:

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        cout << matris[i][j] << "\t";
    }
    cout << endl;
}
```

Çıktı:

```
1   2   3   4
5   6   7   8
9   10  11  12
```

3.8 Dizilerin Avantajları ve Dezavantajları

```
+---------------------------------+-----------------------------------
--+
```

```
| Avantajlar                          | Dezavantajlar
|
+-------------------------------------+-----------------------------------
--+
| Hızlı erişim (indeksle)             | Sabit boyut
|
| Bellek verimliliği                  | Boyut değiştirilemez
|
| Döngüyle kolay işlem                | Ekleme/silme zor
|
| Ardışık bellek kullanımı            | Sınır kontrolü yok (C++'ta)
|
| Çok boyutlu yapı desteği            | Tüm elemanlar aynı tipte olmalı
|
+-------------------------------------+-----------------------------------
--+
```

---

4. LİSTELER (VECTORS)

4.1 Vector Nedir?

C++ dilinde **vector**, dinamik boyutlu bir dizidir. Boyutu çalışma zamanında değiştirilebilir. `<vector>` kütüphanesi ile kullanılır.

Genel Yapı:

```
#include <vector>
vector<veri_tipi> liste_adi;
```

Örnekler:

```
vector<int> sayilar;
vector<double> notlar;
vector<string> isimler;
```

4.2 Vector Tanımlama ve Başlatma

```
// Boş vector
vector<int> v1;

// 5 elemanlı, hepsi 0
vector<int> v2(5);

// 5 elemanlı, hepsi 10
vector<int> v3(5, 10);
```

```
// Başlangıç listesi ile
vector<int> v4 = {10, 20, 30, 40, 50};

// Kopyalama
vector<int> v5(v4);
```

4.3 Vector Elemanlarına Erişim

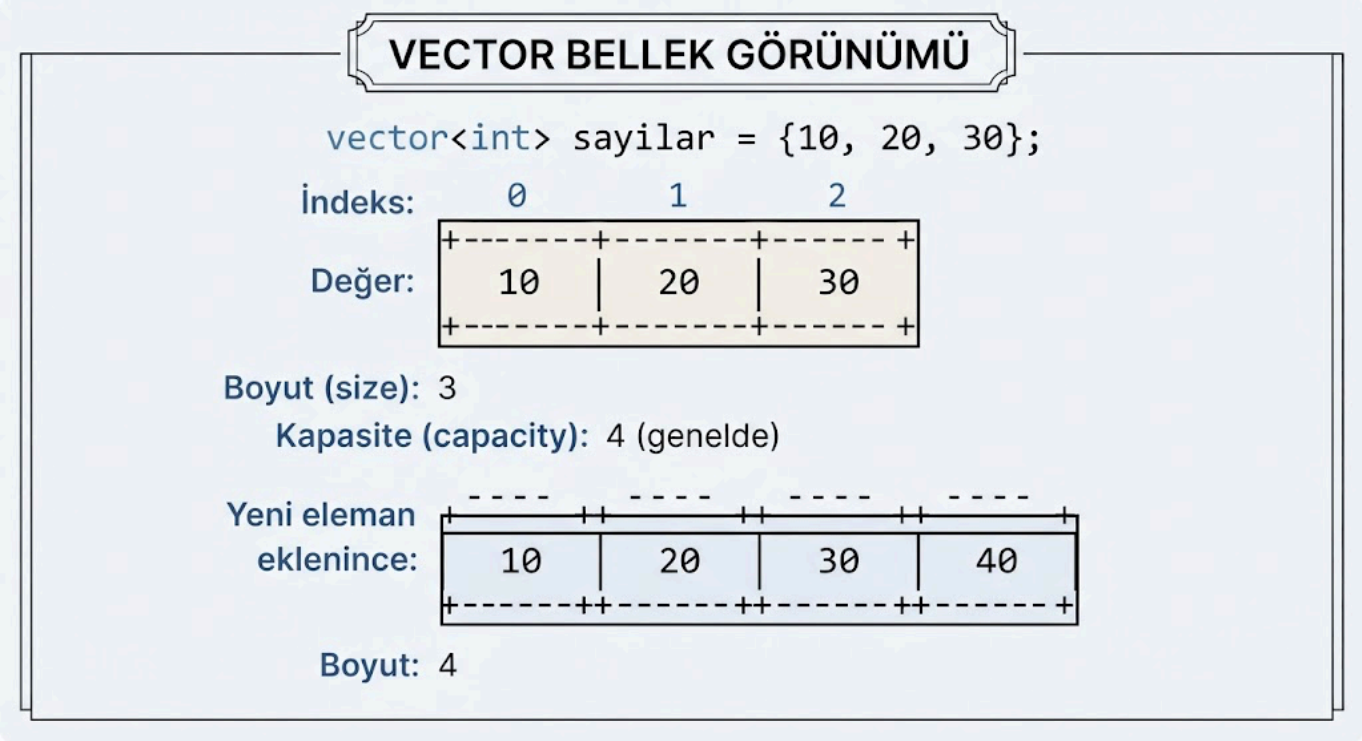
```
vector<int> sayilar = {10, 20, 30, 40, 50};

// İndeksle erişim
cout << sayilar[0];      // 10
cout << sayilar[2];      // 30

// at() ile erişim (sınır kontrolü yapar)
cout << sayilar.at(1);   // 20

// İlk ve son eleman
cout << sayilar.front(); // 10
cout << sayilar.back();  // 50
```

4.4 Vector'ün Bellek Görünümü



4.5 Temel Liste İşlemleri

Eleman Ekleme:

```
vector<int> sayilar;

// Sona ekleme
sayilar.push_back(10);
sayilar.push_back(20);
sayilar.push_back(30);
// sayilar = {10, 20, 30}

// Başa ekleme
sayilar.insert(sayilar.begin(), 5);
// sayilar = {5, 10, 20, 30}

// Belirli bir konuma ekleme
sayilar.insert(sayilar.begin() + 2, 15);
// sayilar = {5, 10, 15, 20, 30}
```

Eleman Silme:

```
vector<int> sayilar = {10, 20, 30, 40, 50};

// Sondan silme
sayilar.pop_back();
// sayilar = {10, 20, 30, 40}

// Belirli bir konumdan silme
sayilar.erase(sayilar.begin() + 1);
// sayilar = {10, 30, 40}

// Belirli bir aralığı silme
sayilar.erase(sayilar.begin(), sayilar.begin() + 2);
// sayilar = {40}

// Tümünü silme
sayilar.clear();
// sayilar = {}
```

Boyut ve Kapasite:

```
vector<int> sayilar = {10, 20, 30};

cout << sayilar.size();      // 3
cout << sayilar.capacity();  // 4 (veya daha fazla)
cout << sayilar.empty();     // 0 (false)
```

Eleman Değiştirme:

```
vector<int> sayilar = {10, 20, 30};
sayilar[1] = 25;        // {10, 25, 30}
sayilar.at(2) = 35;     // {10, 25, 35}
```

4.6 Liste İşlemleri Tablosu

```
+---------------------+-----------------------------------------+
| İşlem               | Açıklama                                |
+---------------------+-----------------------------------------+
| push_back(x)        | Sona eleman ekler                       |
| pop_back()          | Sondan eleman siler                     |
| insert(pos, x)      | Belirli konuma eleman ekler             |
| erase(pos)          | Belirli konumdaki elemanı siler         |
| clear()             | Tüm elemanları siler                    |
| size()              | Eleman sayısını döndürür                |
| capacity()          | Kapasiteyi döndürür                     |
| empty()             | Liste boş mu kontrol eder               |
| front()             | İlk elemanı döndürür                    |
| back()              | Son elemanı döndürür                    |
| at(i)               | i. elemanı döndürür (sınır kontrolü)    |
| begin()             | Başlangıç iteratörü                     |
| end()               | Bitiş iteratörü                         |
| resize(n)           | Boyutu n yapar                          |
| reserve(n)          | Kapasiteyi n yapar                      |
+---------------------+-----------------------------------------+
```

4.7 Vector'de Döngü Kullanımı

Klasik For Döngüsü:

```
vector<int> sayilar = {10, 20, 30, 40, 50};

for (int i = 0; i < sayilar.size(); i++) {
    cout << sayilar[i] << " ";
}
```

Range-based For Döngüsü:

```
for (int eleman : sayilar) {
    cout << eleman << " ";
}
```

İteratör ile:

```
for (auto it = sayilar.begin(); it != sayilar.end(); ++it) {
    cout << *it << " ";
}
```

4.8 Vector'ün Avantajları ve Dezavantajları

```
+----------------------------------+--------------------------------------
--+
| Avantajlar                       | Dezavantajlar
|
+----------------------------------+--------------------------------------
--+
| Dinamik boyut                    | Diziye göre biraz yavaş
|
| Kolay ekleme/silme               | Bellek taşması (reallocation)
|
| Sınır kontrolü (at ile)          | İteratör geçersizliği
|
| Otomatik bellek yönetimi         | Diziye göre daha fazla bellek
|
| Zengin fonksiyon seti            |
|
+----------------------------------+--------------------------------------
--+
```

---

5. DİZİ vs LİSTE KARŞILAŞTIRMASI

```
+---------------------+---------------------+---------------------+
| Özellik             | Dizi (Array)        | Liste (Vector)      |
+---------------------+---------------------+---------------------+
| Boyut               | Sabit               | Dinamik             |
| Tanımlama           | int d[5];           | vector<int> v;      |
| Ekleme              | Yok                 | push_back()         |
| Silme               | Yok                 | pop_back(), erase() |
| Boyut öğrenme       | sizeof/sizeof       | size()              |
| Sınır kontrolü      | Yok                 | at() ile var        |
| Bellek yönetimi     | Manuel              | Otomatik            |
| Kopyalama           | Manuel              | Otomatik            |
| Fonksiyona geçirme  | Pointer             | Değer/Referans      |
| Performans          | Çok hızlı           | Hızlı               |
| Kullanım durumu     | Sabit boyutlu veri  | Değişken boyutlu veri|
+---------------------+---------------------+---------------------+
```

---

6. UYGULAMA: DİZİ VE LİSTE PROBLEMLERİ (C++ DİLİNDE)

6.1 Problem 1: Dizi Elemanlarını Yazdırma

Problem: Kullanıcıdan alınan 5 sayıyı diziye atıp ekrana yazdıran C++ programı.

```cpp
#include <iostream>
using namespace std;

int main() {
    int sayilar[5];

    cout << "5 sayı girin:" << endl;
    for (int i = 0; i < 5; i++) {
        cout << i + 1 << ". sayı: ";
        cin >> sayilar[i];
    }

    cout << "Girdiğiniz sayılar: ";
    for (int i = 0; i < 5; i++) {
        cout << sayilar[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Çalışma Örneği:

```
5 sayı girin:
1. sayı: 10
2. sayı: 20
3. sayı: 30
4. sayı: 40
5. sayı: 50
Girdiğiniz sayılar: 10 20 30 40 50
```

6.2 Problem 2: Dizi Toplamı ve Ortalaması

Problem: Kullanıcıdan alınan N sayının toplamını ve ortalamasını bulan C++ programı.

```cpp
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;
```

```
    int sayilar[100];
    int toplam = 0;

    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> sayilar[i];
        toplam += sayilar[i];
    }

    double ortalama = (double)toplam / n;

    cout << "Toplam: " << toplam << endl;
    cout << "Ortalama: " << ortalama << endl;

    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 5
Sayıları girin:
10 20 30 40 50
Toplam: 150
Ortalama: 30
```

6.3 Problem 3: En Büyük ve En Küçük Eleman

Problem: Dizideki en büyük ve en küçük elemanı bulan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    int sayilar[100];

    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> sayilar[i];
    }

    int en_buyuk = sayilar[0];
    int en_kucuk = sayilar[0];

    for (int i = 1; i < n; i++) {
```

```
        if (sayilar[i] > en_buyuk) {
            en_buyuk = sayilar[i];
        }
        if (sayilar[i] < en_kucuk) {
            en_kucuk = sayilar[i];
        }
    }

    cout << "En büyük: " << en_buyuk << endl;
    cout << "En küçük: " << en_kucuk << endl;

    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 6
Sayıları girin:
45 12 78 23 90 56
En büyük: 90
En küçük: 12
```

6.4 Problem 4: Diziyi Ters Çevirme

Problem: Kullanıcıdan alınan diziyi ters çevirip yazdıran C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    int sayilar[100];

    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> sayilar[i];
    }

    cout << "Ters çevrilmiş dizi: ";
    for (int i = n - 1; i >= 0; i--) {
        cout << sayilar[i] << " ";
    }
    cout << endl;
```

```
    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 5
Sayıları girin:
10 20 30 40 50
Ters çevrilmiş dizi: 50 40 30 20 10
```

6.5 Problem 5: Dizide Arama

Problem: Kullanıcıdan alınan dizide belirli bir elemanın olup olmadığını bulan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n, aranan;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    int sayilar[100];

    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> sayilar[i];
    }

    cout << "Aranacak sayı: ";
    cin >> aranan;

    bool bulundu = false;
    int konum = -1;

    for (int i = 0; i < n; i++) {
        if (sayilar[i] == aranan) {
            bulundu = true;
            konum = i;
            break;
        }
    }

    if (bulundu) {
        cout << aranan << " sayısı " << konum << ". indekste bulundu." <<
endl;
    } else {
        cout << aranan << " sayısı bulunamadı." << endl;
```

```
    }

    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 5
Sayıları girin:
10 25 30 45 50
Aranacak sayı: 30
30 sayısı 2. indekste bulundu.
```

6.6 Problem 6: Diziyi Sıralama (Bubble Sort)

Problem: Kullanıcıdan alınan diziyi küçükten büyüğe sıralayan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    int sayilar[100];

    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> sayilar[i];
    }

    // Bubble Sort
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (sayilar[j] > sayilar[j + 1]) {
                int gecici = sayilar[j];
                sayilar[j] = sayilar[j + 1];
                sayilar[j + 1] = gecici;
            }
        }
    }

    cout << "Sıralı dizi: ";
    for (int i = 0; i < n; i++) {
        cout << sayilar[i] << " ";
    }
    cout << endl;
```

```
    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 6
Sayıları girin:
64 25 12 22 11 90
Sıralı dizi: 11 12 22 25 64 90
```

6.7 Problem 7: Çift ve Tek Sayıları Ayırma

Problem: Dizideki çift ve tek sayıları ayrı ayrı yazdıran C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    int sayilar[100];

    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> sayilar[i];
    }

    cout << "Çift sayılar: ";
    for (int i = 0; i < n; i++) {
        if (sayilar[i] % 2 == 0) {
            cout << sayilar[i] << " ";
        }
    }
    cout << endl;

    cout << "Tek sayılar: ";
    for (int i = 0; i < n; i++) {
        if (sayilar[i] % 2 != 0) {
            cout << sayilar[i] << " ";
        }
    }
    cout << endl;
```

```
    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 7
Sayıları girin:
10 15 22 33 40 55 60
Çift sayılar: 10 22 40 60
Tek sayılar: 15 33 55
```

6.8 Problem 8: İki Diziyi Birleştirme

Problem: İki diziyi birleştirip tek dizi hâlinde yazdıran C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n1, n2;

    cout << "Birinci dizinin eleman sayısı: ";
    cin >> n1;

    int dizi1[100];
    cout << "Birinci diziyi girin:" << endl;
    for (int i = 0; i < n1; i++) {
        cin >> dizi1[i];
    }

    cout << "İkinci dizinin eleman sayısı: ";
    cin >> n2;

    int dizi2[100];
    cout << "İkinci diziyi girin:" << endl;
    for (int i = 0; i < n2; i++) {
        cin >> dizi2[i];
    }

    int birlesik[200];

    for (int i = 0; i < n1; i++) {
        birlesik[i] = dizi1[i];
    }
    for (int i = 0; i < n2; i++) {
        birlesik[n1 + i] = dizi2[i];
    }

    cout << "Birleşik dizi: ";
```

```
    for (int i = 0; i < n1 + n2; i++) {
        cout << birlesik[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Çalışma Örneği:

```
Birinci dizinin eleman sayısı: 3
Birinci diziyi girin:
10 20 30
İkinci dizinin eleman sayısı: 4
İkinci diziyi girin:
40 50 60 70
Birleşik dizi: 10 20 30 40 50 60 70
```

6.9 Problem 9: Vector ile Dinamik Liste

Problem: Kullanıcıdan alınan sayıları vector'e ekleyip, ardından liste işlemlerini gösteren C++ programı.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> sayilar;
    int n, sayi;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> sayi;
        sayilar.push_back(sayi);
    }

    cout << "Liste boyutu: " << sayilar.size() << endl;
    cout << "Liste elemanları: ";
    for (int eleman : sayilar) {
        cout << eleman << " ";
    }
    cout << endl;

    cout << "İlk eleman: " << sayilar.front() << endl;
    cout << "Son eleman: " << sayilar.back() << endl;
```

```
    sayilar.pop_back();
    cout << "Sondan bir eleman silindi. Yeni boyut: " << sayilar.size() <<
endl;

    sayilar.insert(sayilar.begin(), 100);
    cout << "Başa 100 eklendi. İlk eleman: " << sayilar.front() << endl;

    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 4
Sayıları girin:
10 20 30 40
Liste boyutu: 4
Liste elemanları: 10 20 30 40
İlk eleman: 10
Son eleman: 40
Sondan bir eleman silindi. Yeni boyut: 3
Başa 100 eklendi. İlk eleman: 100
```

6.10 Problem 10: Matris Toplama

Problem: İki matrisi toplayan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int satir, sutun;

    cout << "Satır sayısı: ";
    cin >> satir;
    cout << "Sütun sayısı: ";
    cin >> sutun;

    int matris1[10][10], matris2[10][10], toplam[10][10];

    cout << "Birinci matrisi girin:" << endl;
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
            cin >> matris1[i][j];
        }
    }

    cout << "İkinci matrisi girin:" << endl;
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
```

```
            cin >> matris2[i][j];
        }
    }

    // Toplama
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
            toplam[i][j] = matris1[i][j] + matris2[i][j];
        }
    }

    cout << "Matrislerin toplamı:" << endl;
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
            cout << toplam[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Çalışma Örneği:

```
Satır sayısı: 2
Sütun sayısı: 3
Birinci matrisi girin:
1 2 3
4 5 6
İkinci matrisi girin:
7 8 9
10 11 12
Matrislerin toplamı:
8   10  12
14  16  18
```

6.11 Problem 11: Öğrenci Not Sistemi

Problem: N öğrencinin notunu diziye atıp, ortalamayı ve geçen öğrenci sayısını bulan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Öğrenci sayısı: ";
    cin >> n;
```

```
    int notlar[100];
    int toplam = 0;
    int gecen = 0;

    cout << "Notları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> notlar[i];
        toplam += notlar[i];
        if (notlar[i] >= 60) {
            gecen++;
        }
    }

    double ortalama = (double)toplam / n;

    cout << "Sınıf ortalaması: " << ortalama << endl;
    cout << "Geçen öğrenci sayısı: " << gecen << endl;
    cout << "Kalan öğrenci sayısı: " << (n - gecen) << endl;

    return 0;
}
```

Çalışma Örneği:

```
Öğrenci sayısı: 5
Notları girin:
75 82 45 90 58
Sınıf ortalaması: 70
Geçen öğrenci sayısı: 3
Kalan öğrenci sayısı: 2
```

6.12 Problem 12: Frekans Hesaplama

Problem: Bir dizideki her elemanın kaç kez geçtiğini bulan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    int sayilar[100];

    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> sayilar[i];
```

```
    }

    cout << "Frekanslar:" << endl;
    for (int i = 0; i < n; i++) {
        int sayac = 0;
        bool onceki = false;

        // Daha önce sayıldı mı kontrol et
        for (int j = 0; j < i; j++) {
            if (sayilar[j] == sayilar[i]) {
                onceki = true;
                break;
            }
        }

        if (!onceki) {
            for (int j = 0; j < n; j++) {
                if (sayilar[j] == sayilar[i]) {
                    sayac++;
                }
            }
            cout << sayilar[i] << " -> " << sayac << " kez" << endl;
        }
    }

    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 8
Sayıları girin:
10 20 10 30 20 10 40 30
Frekanslar:
10 -> 3 kez
20 -> 2 kez
30 -> 2 kez
40 -> 1 kez
```

6.13 Problem 13: Vector ile Not Ortalaması

Problem: Vector kullanarak öğrenci notlarının ortalamasını bulan C++ programı.

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    vector<int> notlar;
```

```
    int n, notu;

    cout << "Kaç not gireceksiniz? ";
    cin >> n;

    cout << "Notları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> notu;
        notlar.push_back(notu);
    }

    int toplam = 0;
    for (int eleman : notlar) {
        toplam += eleman;
    }

    double ortalama = (double)toplam / notlar.size();

    cout << "Not sayısı: " << notlar.size() << endl;
    cout << "Toplam: " << toplam << endl;
    cout << "Ortalama: " << ortalama << endl;

    return 0;
}
```

Çalışma Örneği:

```
Kaç not gireceksiniz? 5
Notları girin:
80 75 90 85 70
Not sayısı: 5
Toplam: 400
Ortalama: 80
```

6.14 Problem 14: En Çok Tekrar Eden Eleman

Problem: Dizide en çok tekrar eden elemanı bulan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    int sayilar[100];
```

```
    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> sayilar[i];
    }

    int en_cok = sayilar[0];
    int en_cok_sayi = 0;

    for (int i = 0; i < n; i++) {
        int sayac = 0;
        for (int j = 0; j < n; j++) {
            if (sayilar[j] == sayilar[i]) {
                sayac++;
            }
        }
        if (sayac > en_cok_sayi) {
            en_cok_sayi = sayac;
            en_cok = sayilar[i];
        }
    }

    cout << "En çok tekrar eden: " << en_cok << " (" << en_cok_sayi << "
kez)" << endl;

    return 0;
}
```

Çalışma Örneği:

```
Kaç sayı gireceksiniz? 8
Sayıları girin:
10 20 10 30 20 10 40 10
En çok tekrar eden: 10 (4 kez)
```

6.15 Problem 15: İki Dizinin Kesişimi

Problem: İki dizinin ortak elemanlarını bulan C++ programı.

```
#include <iostream>
using namespace std;

int main() {
    int n1, n2;

    cout << "Birinci dizinin eleman sayısı: ";
    cin >> n1;
    int dizi1[100];
    cout << "Birinci diziyi girin:" << endl;
    for (int i = 0; i < n1; i++) {
```

```
        cin >> dizi1[i];
    }

    cout << "İkinci dizinin eleman sayısı: ";
    cin >> n2;
    int dizi2[100];
    cout << "İkinci diziyi girin:" << endl;
    for (int i = 0; i < n2; i++) {
        cin >> dizi2[i];
    }

    cout << "Ortak elemanlar: ";
    for (int i = 0; i < n1; i++) {
        for (int j = 0; j < n2; j++) {
            if (dizi1[i] == dizi2[j]) {
                cout << dizi1[i] << " ";
                break;
            }
        }
    }
    cout << endl;

    return 0;
}
```

Çalışma Örneği:

```
Birinci dizinin eleman sayısı: 5
Birinci diziyi girin:
10 20 30 40 50
İkinci dizinin eleman sayısı: 4
İkinci diziyi girin:
30 40 60 70
Ortak elemanlar: 30 40
```

6.16 Problem 16: Vector ile Dinamik Öğrenci Listesi

Problem: Vector kullanarak öğrenci isimlerini saklayan ve listeleyen C++ programı.

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

int main() {
    vector<string> ogrenciler;
    int n;
    string isim;
```

```
    cout << "Kaç öğrenci gireceksiniz? ";
    cin >> n;
    cin.ignore();  // Buffer temizleme

    cout << "Öğrenci isimlerini girin:" << endl;
    for (int i = 0; i < n; i++) {
        getline(cin, isim);
        ogrenciler.push_back(isim);
    }

    cout << "\nÖğrenci Listesi:" << endl;
    for (int i = 0; i < ogrenciler.size(); i++) {
        cout << i + 1 << ". " << ogrenciler[i] << endl;
    }

    // Silme işlemi
    if (!ogrenciler.empty()) {
        cout << "\nSon öğrenci silindi: " << ogrenciler.back() << endl;
        ogrenciler.pop_back();
    }

    cout << "Güncel liste boyutu: " << ogrenciler.size() << endl;

    return 0;
}
```

Çalışma Örneği:

```
Kaç öğrenci gireceksiniz? 3
Öğrenci isimlerini girin:
Ali Yılmaz
Ayşe Demir
Mehmet Kaya

Öğrenci Listesi:
1. Ali Yılmaz
2. Ayşe Demir
3. Mehmet Kaya

Son öğrenci silindi: Mehmet Kaya
Güncel liste boyutu: 2
```

6.17 Problem 17: Matris Transpozu

Problem: Bir matrisin transpozunu (satır ve sütunların yer değiştirmesi) bulan C++ programı.

```
#include <iostream>
using namespace std;
```

```cpp
int main() {
    int satir, sutun;

    cout << "Satır sayısı: ";
    cin >> satir;
    cout << "Sütun sayısı: ";
    cin >> sutun;

    int matris[10][10], transpoz[10][10];

    cout << "Matrisi girin:" << endl;
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
            cin >> matris[i][j];
        }
    }

    // Transpoz
    for (int i = 0; i < satir; i++) {
        for (int j = 0; j < sutun; j++) {
            transpoz[j][i] = matris[i][j];
        }
    }

    cout << "Transpoz:" << endl;
    for (int i = 0; i < sutun; i++) {
        for (int j = 0; j < satir; j++) {
            cout << transpoz[i][j] << "\t";
        }
        cout << endl;
    }

    return 0;
}
```

Çalışma Örneği:

```
Satır sayısı: 2
Sütun sayısı: 3
Matrisi girin:
1 2 3
4 5 6
Transpoz:
1   4
2   5
3   6
```

6.18 Problem 18: Palindrom Dizi Kontrolü

Problem: Bir dizinin palindrom olup olmadığını kontrol eden C++ programı.

Tanım: Baştan ve sondan okunduğunda aynı olan dizi.

```
#include <iostream>
using namespace std;

int main() {
    int n;

    cout << "Kaç sayı gireceksiniz? ";
    cin >> n;

    int sayilar[100];

    cout << "Sayıları girin:" << endl;
    for (int i = 0; i < n; i++) {
        cin >> sayilar[i];
    }

    bool palindrom = true;

    for (int i = 0; i < n / 2; i++) {
        if (sayilar[i] != sayilar[n - 1 - i]) {
            palindrom = false;
            break;
        }
    }

    if (palindrom) {
        cout << "Dizi palindromdur." << endl;
    } else {
        cout << "Dizi palindrom değildir." << endl;
    }

    return 0;
}
```

Çalışma Örnekleri:

```
Girdi: 1 2 3 2 1   -->  Çıktı: Dizi palindromdur.
Girdi: 1 2 3 4 5   -->  Çıktı: Dizi palindrom değildir.
```

---

7. DİZİ VE LİSTE KULLANIM REHBERİ

```
+-----------------------------------+---------------------------------------
--+
| Durum                             | Öneri
|
```

```
+----------------------------------+--------------------------------------
--+
| Sabit boyutlu veri               | Dizi (Array)
|
| Değişken boyutlu veri            | Vector
|
| Sık erişim, az ekleme/silme      | Dizi
|
| Sık ekleme/silme                 | Vector
|
| Bellek kısıtlı                   | Dizi
|
| Kolaylık ve güvenlik             | Vector
|
| Çok boyutlu veri                 | Dizi (matris)
|
| Dinamik matris                   | vector<vector<int>>
|
+----------------------------------+--------------------------------------
--+
```

---

8. SIK YAPILAN HATALAR

```
+----------------------------------+--------------------------------------
--+
| Hata                             | Doğrusu
|
+----------------------------------+--------------------------------------
--+
| Dizi sınırlarını aşmak           | İndeks 0 ile n-1 arasında olmalı
|
| İndeksi 1'den başlatmak          | İndeks 0'dan başlar
|
| Boyutu yanlış hesaplamak         | sizeof(dizi)/sizeof(dizi[0])
|
| Vector'de push_back unutmak      | Eleman eklemek için push_back
|
| Vector'de indeks sınırı          | at() kullan veya size() kontrol et
|
| Dizi boyutunu değiştirmeye çalışmak| Vector kullan
|
| Başlatılmamış dizi elemanı okumak | Diziyi başlat
|
| String'de cin >> kullanmak       | getline(cin, str) kullan
|
+----------------------------------+--------------------------------------
--+
```

Örnek Hata:

```
// YANLIŞ – Dizi sınırı aşımı
int sayilar[5];
sayilar[5] = 10;  // Hata! Geçerli indeksler 0–4

// DOĞRU
int sayilar[5];
sayilar[4] = 10;  // Son eleman
```

---

9. ÖZET

- **Dizi (Array)**, aynı tipte verilerin bellekte ardışık olarak saklandığı sabit boyutlu yapıdır.
- **Liste (Vector)**, aynı tipte verilerin saklandığı dinamik boyutlu yapıdır.
- Dizi elemanlarına **indeks** ile erişilir; indeksler **0'dan** başlar.
- Dizi boyutu `sizeof(dizi) / sizeof(dizi[0])` ile bulunur.
- **Vector** için `<vector>` kütüphanesi eklenmelidir.
- Temel liste işlemleri: `push_back()`, `pop_back()`, `insert()`, `erase()`, `clear()`, `size()`, `empty()`, `front()`, `back()`, `at()`.
- **İki boyutlu diziler** matris yapıları için kullanılır.
- Dizi ve listeler döngülerle birlikte kullanıldığında çok güçlüdür.
- Bu bölümde 18 farklı dizi/liste problemi C++ diliyle çözülmüştür: yazdırma, toplam, ortalama, en büyük/küçük, ters çevirme, arama, sıralama, çift/tek ayırma, birleştirme, vector işlemleri, matris toplama, öğrenci not sistemi, frekans, en çok tekrar, kesişim, dinamik liste, transpoz ve palindrom.
- Sık yapılan hatalardan kaçınmak için dizi sınırlarına dikkat edilmeli, vector'ün sağladığı kolaylıklardan yararlanılmalıdır.

---

10. DEĞERLENDİRME SORULARI

1. Dizi nedir? Neden ayrı değişkenler yerine dizi kullanılır?
2. Dizi ve vector arasındaki farkları maddeler hâlinde yazınız.
3. Dizi elemanlarına nasıl erişilir? İndeksler neden 0'dan başlar?
4. Bir dizinin boyutu nasıl bulunur?
5. Dizi tanımlama ve başlatma yöntemlerini örneklerle açıklayınız.
6. İki boyutlu dizi nedir? Bir örnekle gösteriniz.
7. Vector'ün temel işlemlerini (ekleme, silme, boyut) örneklerle açıklayınız.
8. `push_back()` ve `pop_back()` fonksiyonlarının görevlerini yazınız.
9. `size()` ve `capacity()` arasındaki fark nedir?
10. Kullanıcıdan alınan 10 sayıyı diziye atıp toplamını bulan C++ programını yazınız.
11. Bir dizideki en büyük ve en küçük elemanı bulan C++ programını yazınız.
12. Bir diziyi ters çeviren C++ programını yazınız.
13. Bir dizide belirli bir elemanın olup olmadığını bulan C++ programını yazınız.
14. Bubble Sort algoritması ile bir diziyi sıralayan C++ programını yazınız.
15. İki matrisi toplayan C++ programını yazınız.
16. Bir matrisin transpozunu bulan C++ programını yazınız.
17. Vector kullanarak dinamik bir öğrenci listesi oluşturan C++ programını yazınız.

18. Bir dizinin palindrom olup olmadığını kontrol eden C++ programını yazınız.
19. Bir dizideki frekansları hesaplayan C++ programını yazınız.
20. İki dizinin kesişimini bulan C++ programını yazınız.
